Transaction Management and ACID Properties

A transaction is a logical unit of database processing executing multiple read and write operations. ACID properties ensure database reliability:

1. Atomicity: All operations complete successfully, or all changes are rolled back (All-or-Nothing).
2. Consistency: Execution in isolation preserves database integrity constraints.
3. Isolation: Intermediate transaction states are invisible to concurrently executing transactions.
4. Durability: Once committed, state changes survive system crashes or power failures via write-ahead logging (WAL).

Concurrency Anomalies and Two-Phase Locking (2PL)

Uncontrolled concurrent executions introduce anomalies: Dirty Read (reading uncommitted data), Non-Repeatable Read (rereading a value modified by another committed transaction), and Phantom Read (rereading a range where new matching rows were inserted).

Two-Phase Locking (2PL) guarantees conflict serializability:
1. Growing Phase: Transaction may acquire locks (Shared S-locks or Exclusive X-locks) but cannot release any lock.
2. Shrinking Phase: Transaction may release locks but cannot acquire new locks.
Strict 2PL holds all exclusive locks until commit or abort, preventing cascading rollbacks.

B-Tree and B+ Tree Index Architectures

Indexes accelerate data retrieval without full table scans. A B+ Tree is a self-balancing search tree where:
1. Internal nodes store only search keys and child pointers, maximizing node fan-out and minimizing tree height (typically 3-4 levels for billions of rows).
2. Leaf nodes store actual data pointers (or full rows in Clustered Indexes) and are linked in a contiguous doubly linked list, enabling O(log n) point lookups and efficient sequential range scans.